UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE

GABRIEL HENRIQUE ROCHA MELO

TRABALHO PRÁTICO - ANÁLISE EMPÍRICA DA COMPLEXIDADE DE ALGORITMOS

NATAL-RN
2026

INTRODUÇÃO

O trabalho prático tem como objetivo analisar empiricamente, utilizando dos conceitos aprendidos em sala de aula, 5 algoritmos diferentes, sendo 2 algoritmos de busca(busca sequencial e busca binária) e 3 algoritmos de ordenação(bubblesort, mergesort e quicksort) através da linguagem de programação C, utilizando de bibliotecas internas da linguagem e de softwares externos para a coleta de dados de execução temporal das funções implementadas e sua visualização em gráficos individuais utilizando do gnuplot. Com os dados coletados e armazenados em um estrutura de dados(orientação a objetos) e os gráficos criados, foi feita a análise comparando o seu comportamento visualizado através da coleta de dados empírica e seu comportamento teórico em suas funções de complexidade respectivas.

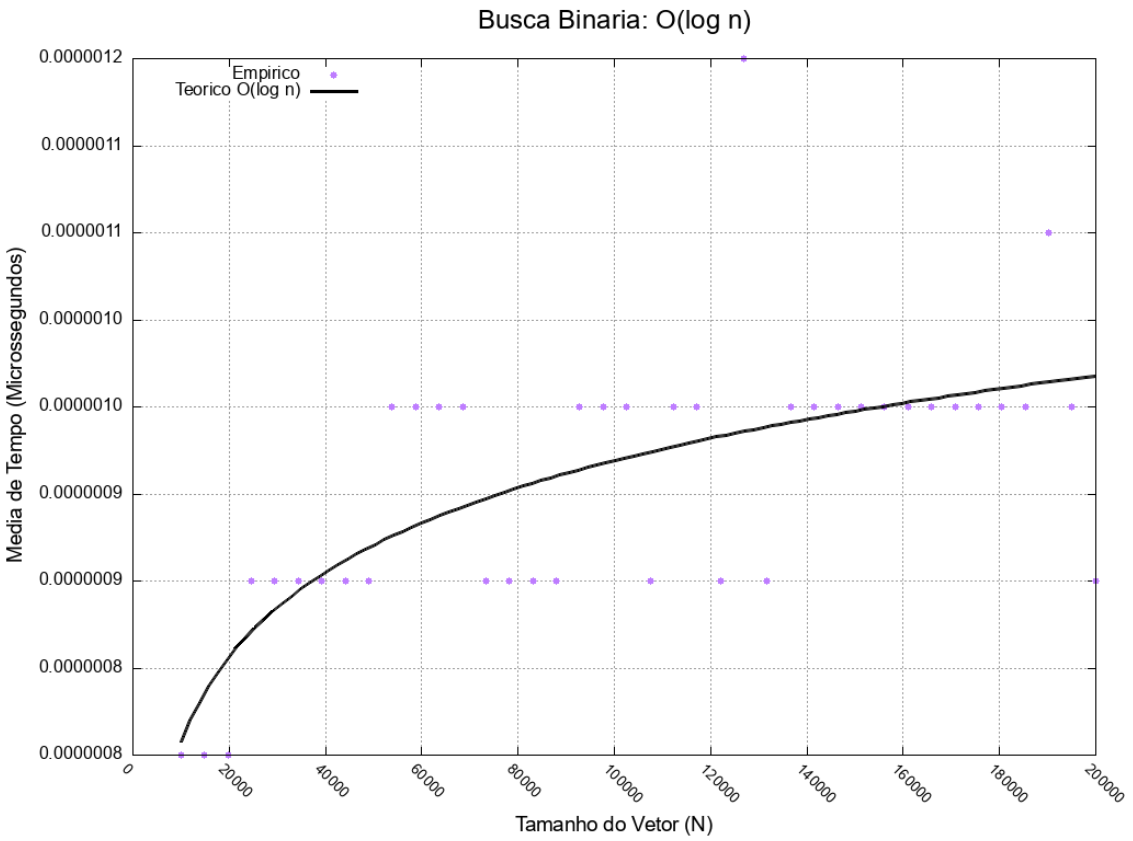
METODOLOGIA

- **Ferramentas utilizadas:** Foram utilizadas como ferramentas para o desenvolvimento do trabalho prático o compilador GCC(GNU Compiler Collection), o editor de código-fonte gratuito Visual Studio Code(VS Code), GitHub Desktop para importações e alterações necessárias de arquivos pertinentes ao projeto e do software Gnuplot para visualização dos gráficos de comportamento.

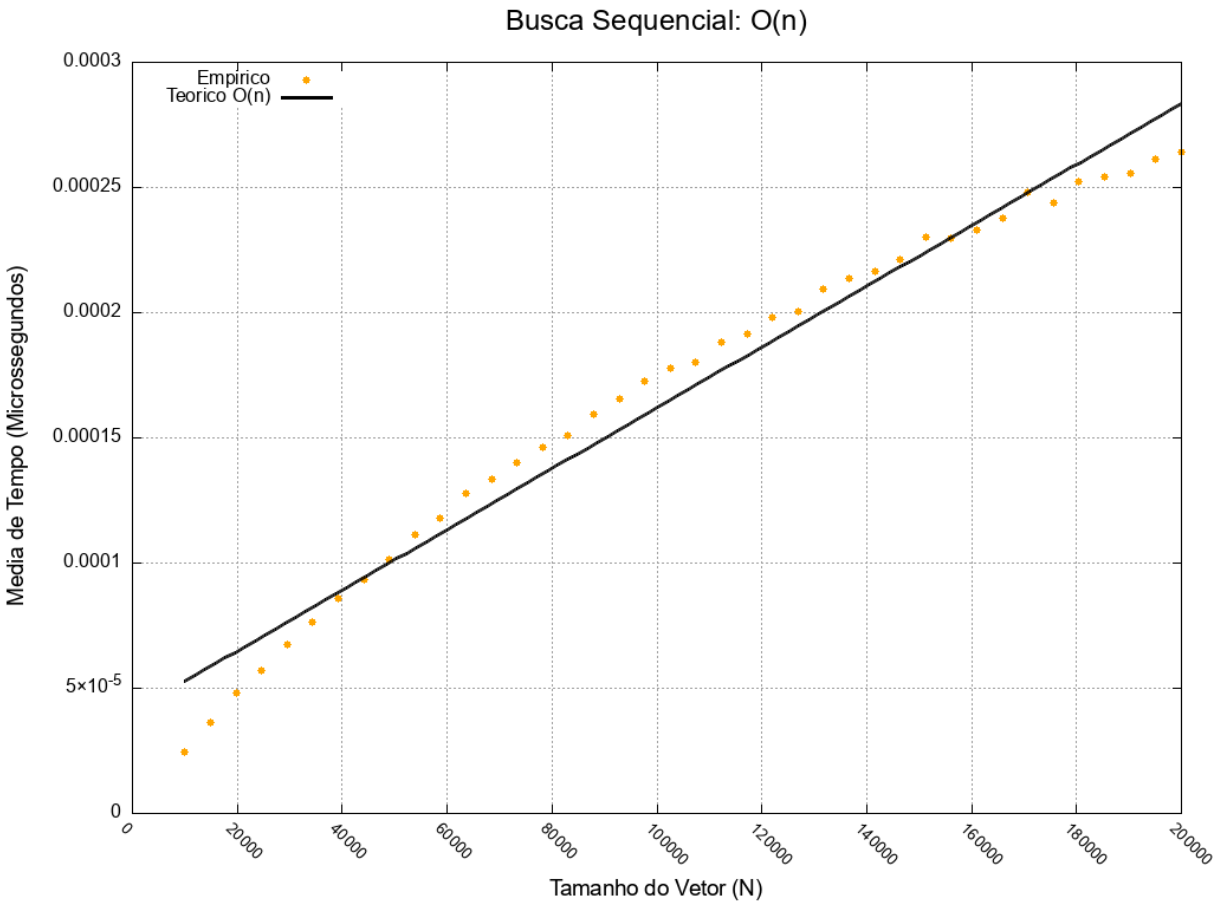
- **Abordagem de desenvolvimento:** Para o desenvolvimento do trabalho prático e por fim a realização da análise empírica, foi inicialmente declarado um vetor vazio de tamanho 200.000 que seria preenchido com valores aleatórios a cada nova iteração com base no tamanho do vetor em específico nessa iteração(variável n_testes). Esse mesmo vetor seria então compartilhado para os 5 algoritmos selecionados e utilizando da biblioteca <time.h>, nativa da linguagem C, para a coleta do tempo de execução dos algoritmos, sendo então posteriormente armazenados em um struct o tamanho do vetor naquela iteração e os tempos de execução dos 5 algoritmos. Esse procedimento seria então executado um número n de vezes, com vetores diferentes a cada iteração, para que se possa adquirir, como resultado, uma análise de comportamento mais precisa desses algoritmos em um grande volume de dados. Vale ressaltar que a coleta temporal dos algoritmos de busca binária, merge sort, quick sort e busca sequencial foram realizadas em um loop e o tempo de execução médio coletado(tempo de execução total dividido pelo total de iterações em seus loops específicos), já que a função clock não é muito precisa para a alta velocidade de execução desses algoritmos(tempos marcados como 'zero' ou valores muito imprecisos) para que em apenas uma única iteração a coleta do tempo de execução seja satisfatória. A única exceção é o bubble sort já que ele é muito lento para um volume grande de dados sem ordenação(médio e pior caso são ambos O(n^2)), levando a um tempo de 5 - 10 minutos para que a compilação e execução do código seja finalizada em uma coleta de apenas 40 tempos de execução. Por fim, com os dados coletados e armazenados no struct, foi criado um arquivo .csv onde esses dados foram guardados e posteriormente utilizados na plotagem de 5 gráficos para cada algoritmo utilizando o software Gnuplot.

- **Organização do código:** Devido a modularização, o trabalho prático foi dividido em 5 arquivos .c: o arquivo analise.c(arquivo principal) e outros 4 arquivos .c contendo as funções pertinentes ao trabalho, como as funções de busca e a função de plotagem de gráfico. Foi criado também 4 arquivos .h(bibliotecas autorais) com todas elas contendo os cabeçalhos das funções implementadas nos 4 arquivos .c.

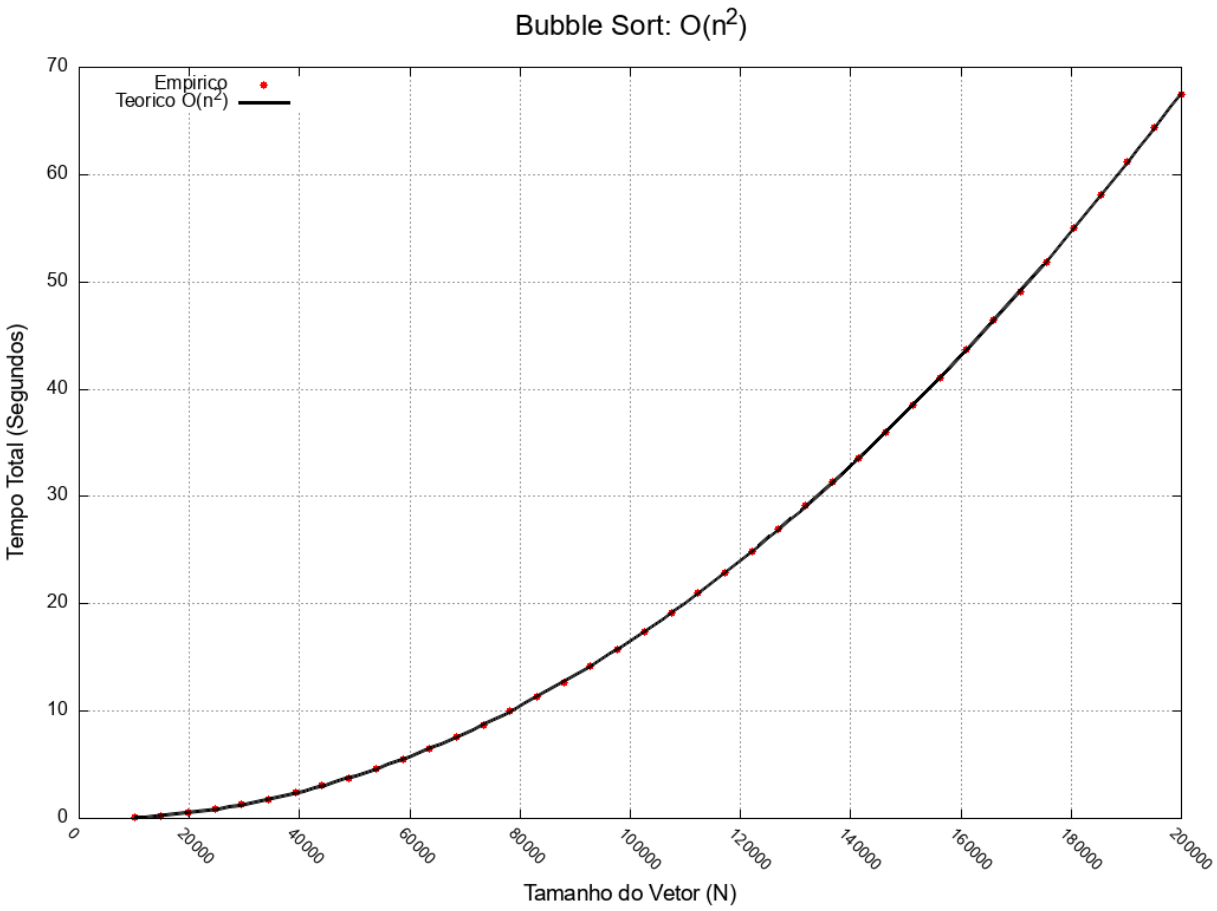
RESULTADOS



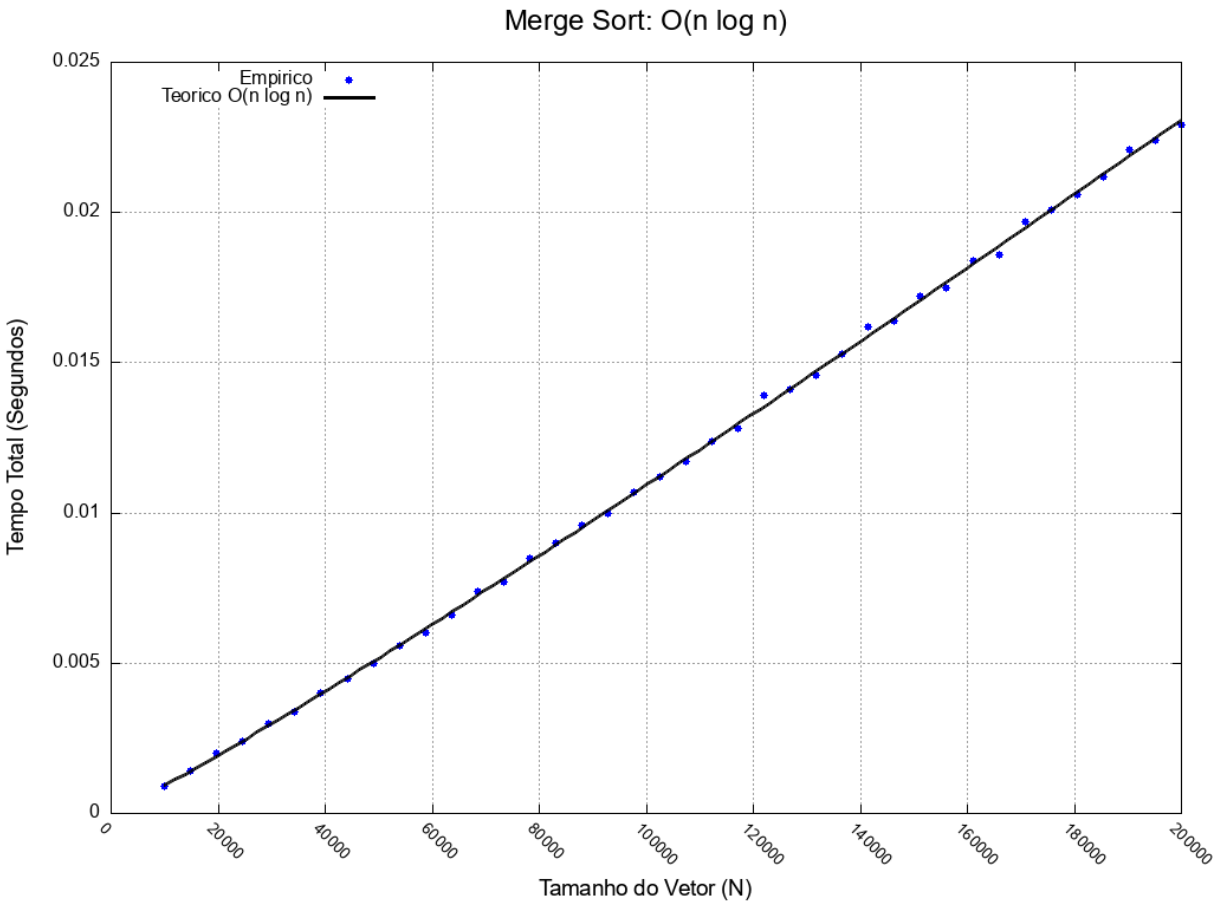
- Embora os pontos violetas, que representam os tempos de execução do algoritmo de busca binária até encontrar a chave nos vetores, não sigam à risca a linha da função de complexidade teórica(linha preta), é perceptível como o comportamento dela se assemelha a uma função logarítmica pela sua velocidade de execução no eixo y que varia muito pouco, quase irrelevante, entre cada execução, provando sua alta velocidade em encontrar a chave em um grande volume dados, fazendo significativamente menos comparações que a busca sequencial(O(n)), dividindo sucessivamente o vetor pela metade a cada iteração. O motivo dos pontos violetas não desenharem uma linha curvada como a teórica se deve tanto ao baixo volume de dados(n execuções do for principal) quanto a ruídos do proprio sistema operacional e a imprecisão da função clock da biblioteca <time.h> da linguagem C.



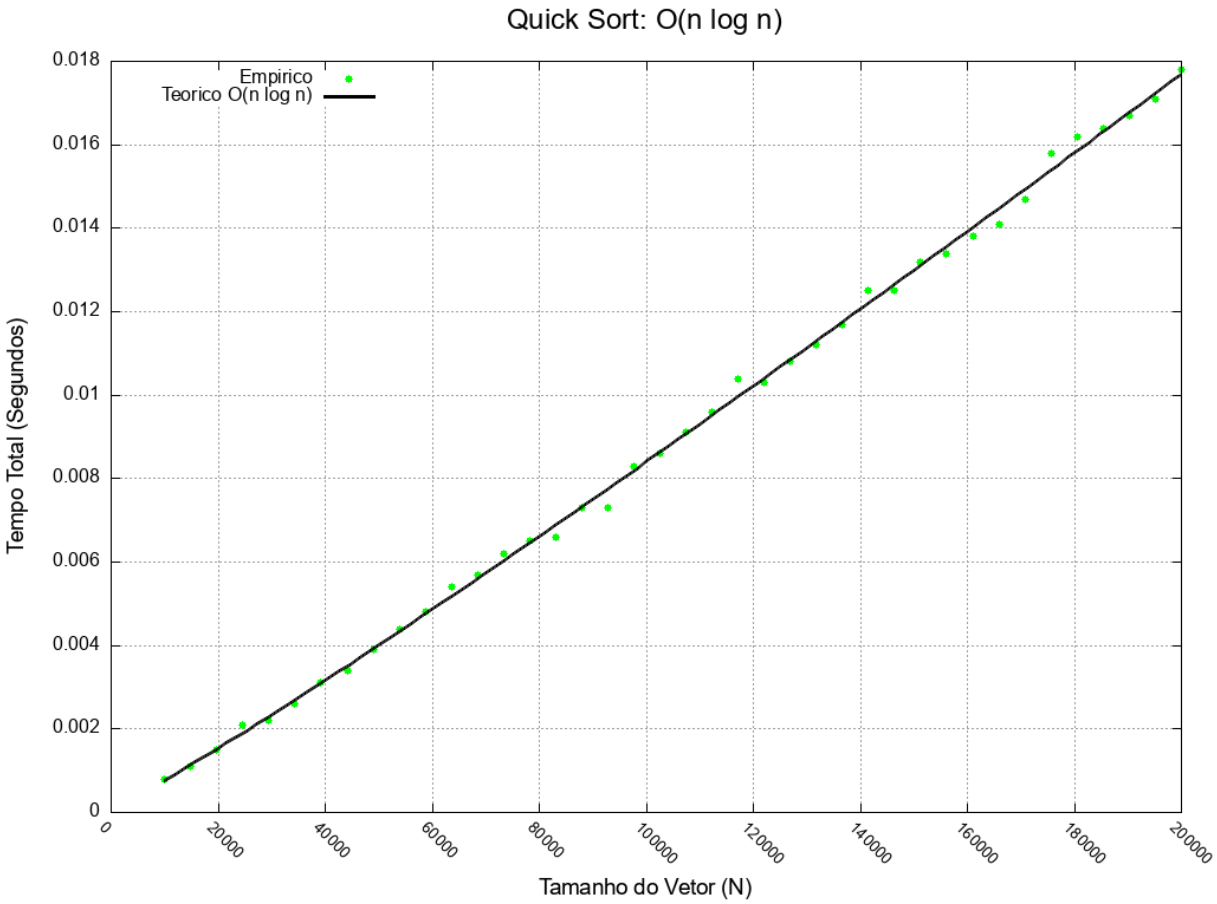
- A busca sequencial trata-se do grau de complexidade linear, ou seja, no pior caso, ela irá comparar n vezes, onde n é o tamanho do vetor. Isso se comprova pelo comportamento do gráfico acima, onde, conforme o tamanho do vetor vai aumentando, a chave aleatória a ser buscada pode estar em posições cada vez mais elevadas, resultando em um tempo de execução cada vez maior, porém ainda pequeno devido a própria simplicidade do algoritmo. Embora os dados coletados não representem um reta exata como na função de complexidade teórica(linha preta), o gráfico ainda pode ser considerado de complexidade o(n) porque cada elemento extra adicionado ao vetor representa, no pior caso, exatamente uma unidade de tempo extra (uma comparação a mais).



- O bubblesort é o gráfico mais fiel o possível à função de complexidade teórica(linha preta) e também o mais lento dentre todos os 5 algoritmos selecionados, já que ele percorre o vetor várias vezes e, a cada passagem, "empurra" o maior elemento para a sua posição correta no final, trocando elementos adjacentes que estão fora de ordem, tornando-o ineficiente para grandes volumes de dados. Isso é comprovado pelo fato de cada tempo de execução(pontos vermelhos) se alinharem quase perfeitamente a função de complexidade teórica, formando um gráfico de uma função quadrática(O(n^2), devido aos laços de repetição aninhados), e seus tempos de execução total(eixo y) serem medidos em segundos e não em microssegundo, nanosegundo ou milisegundo, demonstrando uma relação diretamente proporcional ao volume de dados, ou seja, quanto maior o volume de dados, maior o tempo de execução necessário.



- O algoritmo mergesort, possui grau de complexidade O(n*log(n)), já que baseia-se na estratégia de divisão e conquista, dividindo o vetor recursivamente em duas metades até que cada subvetor tenha apenas um elemento, para então em seguida, ele intercalar esses subvetores de forma ordenada, fazendo a função crescer lentamente e de forma lineritimica. Embora não pareça, o gráfico gerado pelos pontinhos azuis de fato representam um crescimento linearítmico, pois em um baixo volume de dados(como os 40 dados de tempo de execução acima que foram coletados para a plotagem do gráfico), a função de complexidade se assemelha muito a uma função linear. O'que a diferencia do bubble sort, por exemplo, é seu tempo de execução muito curto para volumes massivos de dados, crescendo muito lentamente, possuindo uma diferença quase insignificante para cada tempo de execução, embora, obviamente, o tempo de execução cresça para conjuntos de dados cada vez maiores devido a natureza linearítmica da função.



- O quicksort possui o mesmo grau de complexidade do mergesort(O(n*log(n)), já que ao escolher um elemento como pivô o vetor é organizado de modo que elementos menores que o pivô fiquem à esquerda e maiores à direita, repetindo esse processo de forma recursiva até que array esteja ordenado. Para volumes massivos de dados, seu gráfico de comportamento é idêntico ao do mergesort, mas o que os diferencia, nessa coleta de dados em específico, é seu tempo de execução. Embora o mergesort possua grau de complexidade igual para todos os casos(O(n*log(n)), tornado-o o mais estável, o quicksort, se escolhido bem o pivô(aleatório, como no algoritmo implementado, ou mediana de 3), ordena os valores sem precisar de arrays auxiliares, trocando os elementos dentro do próprio array, gastando menos tempo e memória do que o mergesort que precisa de arrays auxiliares temporários para cada etapa da ordenação, consumindo muito tempo de execução e memória para realizar a operação. Isso é evidenciado pelos tempos de execução no gráfico do quicksort um pouco mais curtos se comparados aos tempos de execução do gráfico do mergesort. Porém, caso o pivô não seja bem escolhido ou o vetor já esteja ordenado, a função e o gráfico de complexidade do quicksort acaba se tornando o mesmo do bubble sort(O(n^2)).

CONCLUSÃO

Por fim, é notável que todo o conhecimento adquirido no decorrer da unidade 1 e implementado ao longo do desenvolvimento do trabalho prático, serão de grande utilidade não só para futuros projetos acadêmicos, mas os projetos de âmbito pessoal também, já que as habilidades adquiridas, principalmente de análise de complexidade e comportamento assintótico serão de grande utilidade no mundo profissional e acadêmico, o'que me ajudará a escolher a ferramenta certa para o problema certo. A análise empírica também me permitiu confrontar a teoria da complexidade assintótica com as limitações físicas do hardware e do sistema operacional, revelando que a eficiência de um software não depende apenas do modelo matemático, mas também de uma gestão consciente de recursos como CPU e memória.